***Student name* :** Jalapati Siva Krishna Charan
• Register number :1925260228
• Compiler and version (e.g., GCC 13): *GCC and version 14.2.0*

TASK-1 Understand the Problem and Identify the Data

Electricity Billing System (4 Marks)

1. Inputs

The program requires the following input data:

Previous Reading: The electricity meter reading recorded in the previous month. It must be a non-negative integer.

Current Reading: The electricity meter reading recorded in the current month. It must be greater than or equal to the previous reading.

Household Number: A unique identification number for each household. It can be stored as an integer or a string.

Category: The type of electricity consumer, such as domestic or commercial. It determines the applicable tariff rates.

2. Outputs

The program calculates and displays the following values:

Units Consumed: Calculated by subtracting the previous reading from the current reading. Units = Current Reading - Previous Reading

Energy Charge: The amount charged for the electricity units consumed according to the applicable tariff.

Fixed Charge: A fixed amount charged based on the consumer category.

Total Bill: The sum of the energy charge and fixed charge. Total Bill = Energy Charge + Fixed Charge

3. Constants

Constants are values that do not change while the program runs.

Tariff Rates: The electricity price per unit for each consumption slab.

Fixed Charges: The fixed charge applicable to each consumer category.

Slab Limits: The unit limits used to calculate charges at different rates.

These values should be declared as constants because they remain unchanged during a billing calculation.

4. Data Types and Validations

Integer: Used for previous reading, current reading, and units consumed because meter readings are usually whole numbers.

String: Used for the household number and consumer category when they contain letters or leading zeros.

Float/Double: Used for energy charges, fixed charges, and the total bill because monetary calculations may involve decimal values.

Validation rules:

Meter readings must not be negative.

The current reading must be greater than or equal to the previous reading.

Units consumed must not be negative.

Charges and the total bill must not be negative.

The total bill must equal the energy charge plus the fixed charge.

5. Assumptions

Meter readings are provided in whole units.

The tariff rates and fixed charges are predefined.

Each household belongs to a valid consumer category.

The meter readings are accurate.

Taxes, subsidies, and additional charges are excluded unless specified.

6. Alternative Concept and Complexity Analysis

If meter readings are stored as text instead of integers:

Conversion: Each reading must be converted from text to an integer before subtraction. For a reading containing d digits, conversion takes **O(d)** time.

Comparison: Comparing two integer readings takes **O(1)** time for fixed-size integers. Comparing text readings takes up to **O(d)** time.

Space: An integer uses a fixed amount of memory, typically 4 bytes. A text reading requires approximately O(d) space, depending on the encoding and storage format.

Conclusion: Storing meter readings as integers is generally more efficient for arithmetic calculations, while strings are useful when leading zeros must be preserved.

TASK-2

INPUT ::

```
#include <stdio.h>
#include <limits.h>

/* The meter register is a 16-bit counter */
typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current)
{
   return (meter_t)(current - previous);
}

int main(void)
{
   printf("sizeof(meter_t) = %zu bytes, maximum reading = %u\n",
         sizeof(meter_t), (unsigned)USHRT_MAX);

   printf("Consumption: %u units\n", consumption(1200, 1530));
   printf("Consumption: %u units\n", consumption(65400, 120));

   return 0;
}
```

OUTPUT ::

Output

```
sizeof(meter_t) = 2 bytes, maximum reading = 65535
Consumption: 330 units
Consumption: 256 units
```

TASK-3

```
#include <stdio.h>

#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450
#define FIXED_CHARGE_PAISE 5000L

long compute_bill(int units)
{
   long paise = FIXED_CHARGE_PAISE;

   if (units > SLAB2_UNITS)
      paise += (long)(units - SLAB2_UNITS) * RATE3_PAISE;

   if (units > SLAB1_UNITS)
   {
      int slab2 = units > SLAB2_UNITS
                ? SLAB2_UNITS - SLAB1_UNITS
                : units - SLAB1_UNITS;

      paise += (long)slab2 * RATE2_PAISE;
   }
```

```
    if (units > 0)
        paise += (long)(units < SLAB1_UNITS
                    ? units : SLAB1_UNITS) * RATE1_PAISE;

    return paise;
}

int main(void)
{
    int tests[] = {0, 100, 150, 200, 350};
    int n = sizeof tests / sizeof tests[0];

    for (int i = 0; i < n; i++)
    {
        long p = compute_bill(tests[i]);
        printf("%3d units -> Rs %ld.%02ld\n",
               tests[i], p / 100, p % 100);
    }

    return 0;
}
```

Output ::

```
Output

  0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 162.50
200 units -> Rs 275.00
350 units -> Rs 950.00
```

TASK-4

```
#include <stdio.h>

#define RUPEES_TO_PAISE(r) ((r) * 100)
#define SQUARE(x) ((x) * (x))
#define MAX(a, b) ((a) > (b) ? (a) : (b))

static inline int square(int x)
{
    return x * x;
}
```

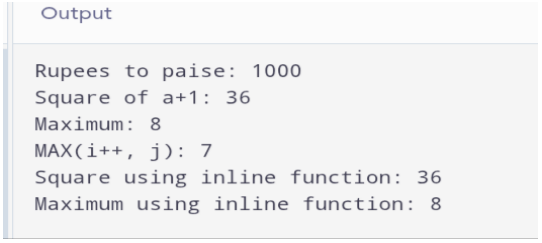
```
static inline int max_value(int a, int b)
{
    return (a > b) ? a : b;
}

int main(void)
{
    int a = 5, b = 8;
    int i = 4, j = 7;

    printf("Rupees to paise: %d\n", RUPEES_TO_PAISE(10));
    printf("Square of a+1: %d\n", SQUARE(a + 1));
    printf("Maximum: %d\n", MAX(a, b));
    printf("MAX(i++, j): %d\n", MAX(i++, j));
    printf("Square using inline function: %d\n", square(a + 1));
    printf("Maximum using inline function: %d\n", max_value(a, b));

    return 0;
}
```

OUTPUT::



TASK-5

```
#include <stdio.h>

#define RATE_PAISE 230

void calculate_bill(int units);

void calculate_bill(int units)
{
    int bill = units * RATE_PAISE;

#ifdef DEBUG
    printf("DEBUG: Units = %d\n", units);
    printf("DEBUG: Bill in paise = %d\n", bill);
#endif

    printf("Total bill = Rs. %.2f\n", bill / 100.0);
```

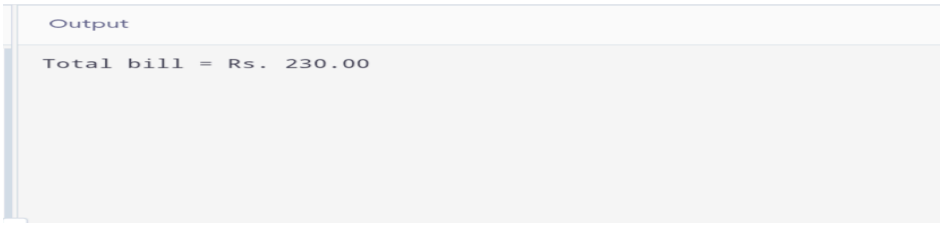
```
}

int main(void)
{
    int units = 100;

    calculate_bill(units);

    return 0;
}
```

Output ::



TASK-6

```
#include <stdio.h>

int main(void)
{
    int units;
    long meter_number = 123456L;
    long long total_paise = 16250LL;
    float temperature = 28.5f;
    double voltage = 230.75;
    char grade = 'A';
    char name[30];

    printf("Enter student name: ");
    if (scanf("%29s", name) != 1)
    {
        printf("Invalid name input.\n");
        return 1;
    }

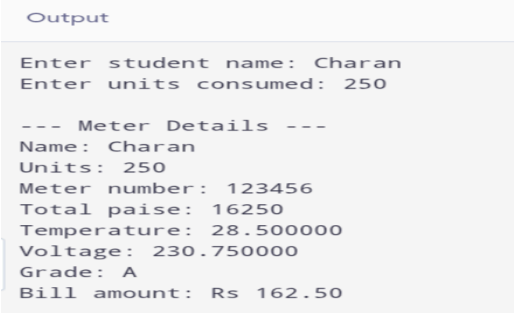
    printf("Enter units consumed: ");
    if (scanf("%d", &units) != 1 || units < 0)
    {
        printf("Invalid units. Enter a non-negative integer.\n");
        return 1;
    }
```

```
    printf("\n--- Meter Details ---\n");
    printf("Name: %s\n", name);
    printf("Units: %d\n", units);
    printf("Meter number: %ld\n", meter_number);
    printf("Total paise: %lld\n", total_paise);
    printf("Temperature: %f\n", temperature);
    printf("Voltage: %lf\n", voltage);
    printf("Grade: %c\n", grade);

    printf("Bill amount: Rs %lld.%02lld\n",
           total_paise / 100, total_paise % 100);

    return 0;
}
```

Output ::



```
Output

Enter student name: Charan
Enter units consumed: 250

--- Meter Details ---
Name: Charan
Units: 250
Meter number: 123456
Total paise: 16250
Temperature: 28.500000
Voltage: 230.750000
Grade: A
Bill amount: Rs 162.50
```

TASK-7

```
#include <stdio.h>
#include <limits.h>

int main(void)
{
    int units = 199;
    int price = 230;
    double charge = 16.257;
    long large_value = 3000000000L;
    float f = 0.1f;
    int total_paise;
    int rounded_paise;
    int i;

    printf("--- Unexpected Results ---\n");

    printf("1. Integer division: %d\n", units / 100);
    printf("2. Integer multiplication: %d\n", units * price / 100);
```

```
    printf("3. Float precision: %.20f\n", f);

    printf("\n--- Type Conversion ---\n");
    printf("Implicit conversion: %.2f\n", (double)units / 100);
    printf("Explicit cast to int: %d\n", (int)charge);

    printf("Large long value: %ld\n", large_value);
    printf("Casting large long to int: %d\n", (int)large_value);

    rounded_paise = (int)(charge * 100 + 0.5);
    printf("\nRounded charge: %d paise\n", rounded_paise);
    printf("Charge: Rs %d.%02d\n",
           rounded_paise / 100, rounded_paise % 100);

    total_paise = 0;
    for (i = 0; i < 1000; i++)
        total_paise += 10;

    printf("\nInteger total for 1000 bills: %d paise\n",
           total_paise);

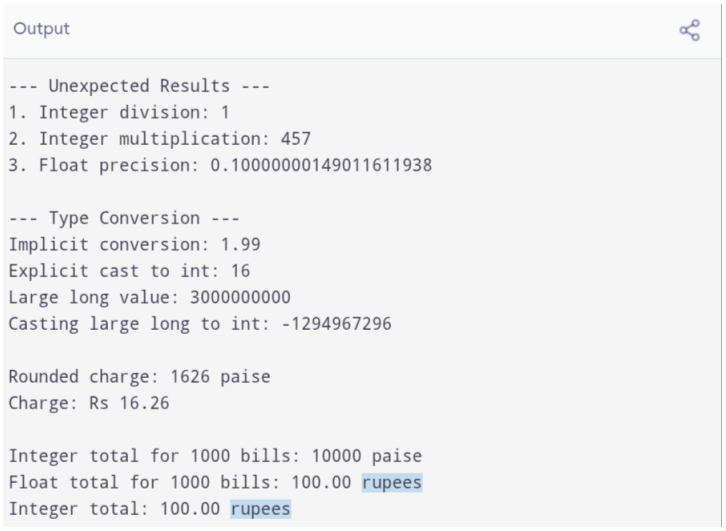
    {
        float total = 0.0f;

        for (i = 0; i < 1000; i++)
            total += 0.10f;

        printf("Float total for 1000 bills: %.2f rupees\n",
               total);
        printf("Integer total: %.2f rupees\n",
               total_paise / 100.0);
    }

    return 0;
}
```

Output::



TASK-8

```
#include <stdio.h>

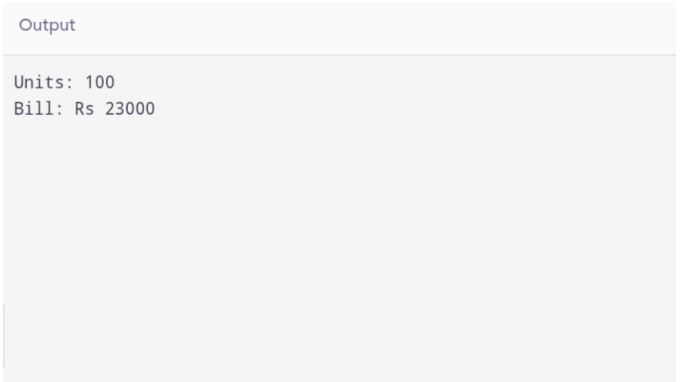
#define RATE 230

int main(void)
{
    int units = 100;
    int bill = units * RATE;

    printf("Units: %d\n", units);
    printf("Bill: Rs %d\n", bill);

    return 0;
}
```

OUTPUT::



TASK-9

```
#include <stdio.h>
```

```c
#include <stdlib.h>
#include <string.h>

#define MAX_READING 65535

double calculate_bill(unsigned int units)
{
    if (units <= 100)
        return units * 1.50;
    else if (units <= 200)
        return 150 + (units - 100) * 2.00;
    else
        return 350 + (units - 200) * 3.00;
}

void test_bill(const char *name, long previous, long current)
{
    unsigned int units;
    double bill;

    printf("\nTest: %s\n", name);
    printf("Previous: %ld, Current: %ld\n", previous, current);

    if (previous < 0 || previous > MAX_READING ||
        current < 0 || current > MAX_READING)
    {
        printf("Result: FAIL - Invalid reading\n");
        return;
    }

    if (current >= previous)
        units = (unsigned int)(current - previous);
    else if (previous == MAX_READING)
        units = (unsigned int)(MAX_READING - previous + 1 + current);
    else
    {
        printf("Result: PASS - Invalid decrease rejected\n");
        return;
    }

    bill = calculate_bill(units);

    printf("Units consumed: %u\n", units);
    printf("Bill amount: Rs %.2f\n", bill);
    printf("Result: PASS\n");
}

int main(void)
```

```
{
    char input[30];
    char *end;
    long value;

    printf("TASK 9 - BILLING PROGRAM TESTS\n");

    test_bill("Normal slab 1", 0, 50);
    test_bill("Normal slab 2", 0, 150);
    test_bill("Normal slab 3", 0, 250);
    test_bill("Boundary 100 units", 0, 100);
    test_bill("Boundary 200 units", 0, 200);
    test_bill("Zero units", 100, 100);
    test_bill("Meter rollover", 65535, 0);
    test_bill("Negative reading", 0, -5);
    test_bill("Invalid decrease", 500, 400);

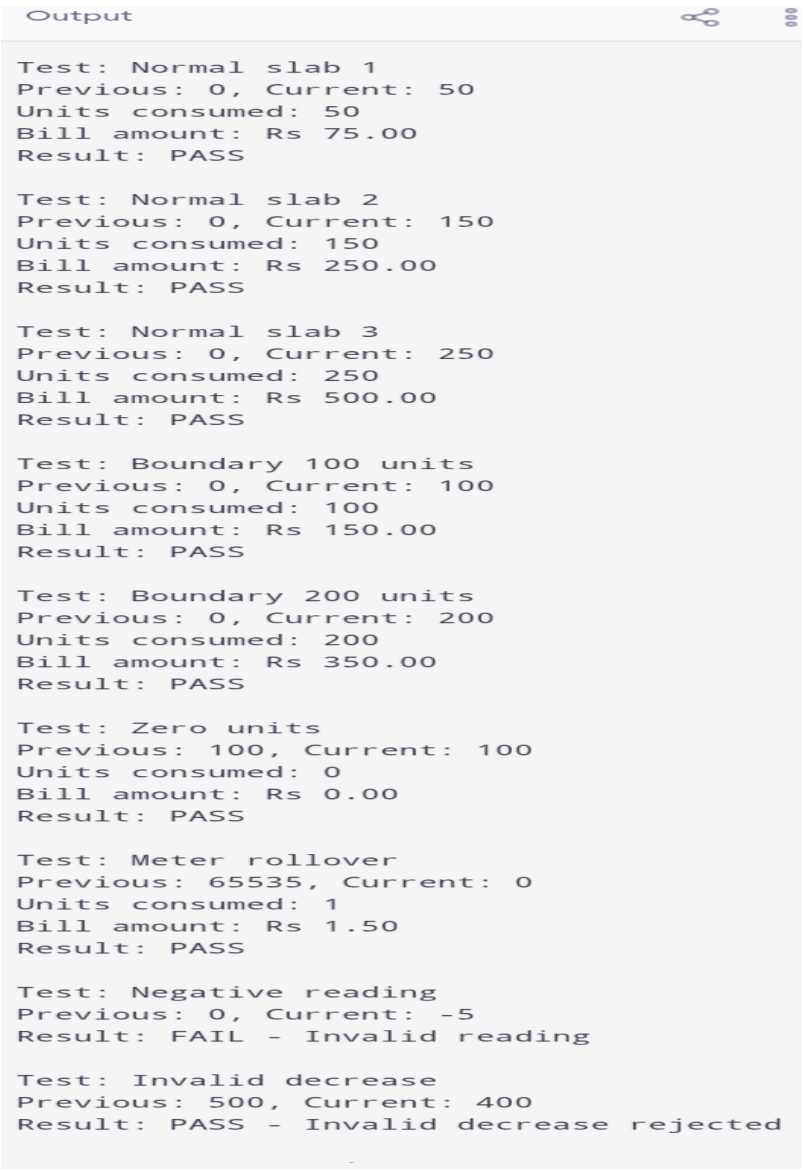
    printf("\nTest: Letters as input\n");
    printf("Enter letters (abc): ");
    if (fgets(input, sizeof(input), stdin) != NULL)
    {
        value = strtol(input, &end, 10);

        if (end == input || (*end != '\n' && *end != '\0'))
            printf("Result: PASS - Invalid letters rejected\n");
        else
            printf("Result: FAIL - Input was numeric\n");
    }

    printf("\nFIVE BILL TOTAL\n");
    printf("Bill 1 (50 units): Rs %.2f\n", calculate_bill(50));
    printf("Bill 2 (100 units): Rs %.2f\n", calculate_bill(100));
    printf("Bill 3 (150 units): Rs %.2f\n", calculate_bill(150));
    printf("Bill 4 (200 units): Rs %.2f\n", calculate_bill(200));
    printf("Bill 5 (250 units): Rs %.2f\n", calculate_bill(250));
    printf("Expected total: Rs 1425.00\n");

    return 0;
}
```

Output::



TASK-10

```
#include <stdio.h>
#include <limits.h>

#define BILL_PAISE 16250LL

int main(void)
{
    int n, i;
    long long paise_total;
    float float_total;
    double double_total;

    int counts[] = {4000, 60000, 800000};
```

```
    printf("DATA TYPE SIZE AND RANGE\n");
    printf("sizeof(int) = %zu bytes\n", sizeof(int));
    printf("sizeof(long) = %zu bytes\n", sizeof(long));
    printf("sizeof(long long) = %zu bytes\n", sizeof(long long));
    printf("sizeof(float) = %zu bytes\n", sizeof(float));
    printf("sizeof(double) = %zu bytes\n", sizeof(double));

    printf("INT_MAX = %d\n", INT_MAX);
    printf("LONG_MAX = %ld\n", LONG_MAX);
    printf("LLONG_MAX = %lld\n\n", LLONG_MAX);

    for (i = 0; i < 3; i++)
    {
        n = counts[i];

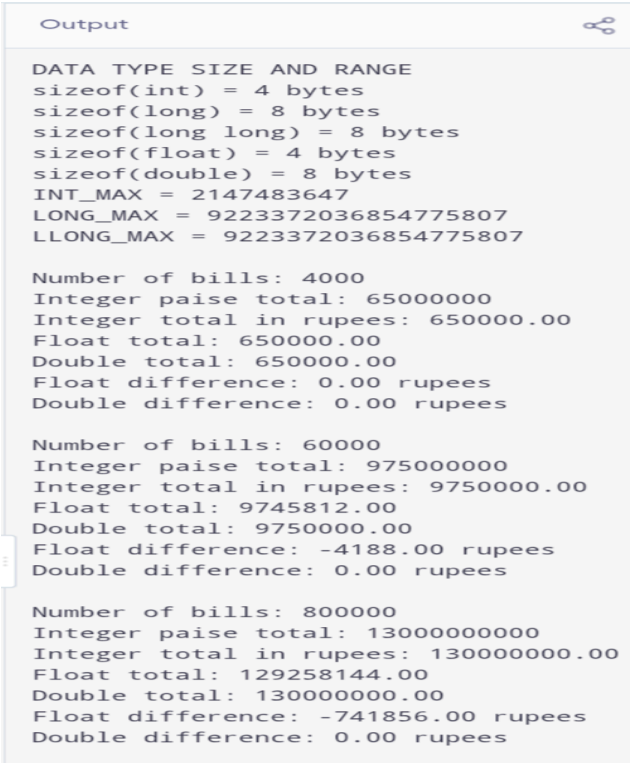
        paise_total = (long long)n * BILL_PAISE;
        float_total = 0.0f;
        double_total = 0.0;

        {
            int j;
            for (j = 0; j < n; j++)
            {
                float_total += 162.50f;
                double_total += 162.50;
            }
        }

        printf("Number of bills: %d\n", n);
        printf("Integer paise total: %lld\n", paise_total);
        printf("Integer total in rupees: %.2f\n",
               paise_total / 100.0);
        printf("Float total: %.2f\n", float_total);
        printf("Double total: %.2f\n", double_total);
        printf("Float difference: %.2f rupees\n",
               (double)float_total - paise_total / 100.0);
        printf("Double difference: %.2f rupees\n\n",
               double_total - paise_total / 100.0);
    }

    return 0;
}
```

OUTPUT



TASK-11

```
#include <stdio.h>
#include <limits.h>

#define MAX_READING 65535U
#define RATE1_PAISE 150
#define RATE2_PAISE 200
#define RATE3_PAISE 300

long long calculate_bill(unsigned int units)
{
    long long bill;

    if (units <= 100)
        bill = (long long)units * RATE1_PAISE;
    else if (units <= 200)
        bill = 100LL * RATE1_PAISE
             + (long long)(units - 100) * RATE2_PAISE;
    else
        bill = 100LL * RATE1_PAISE
             + 100LL * RATE2_PAISE
             + (long long)(units - 200) * RATE3_PAISE;

    return bill;
}
```

```
int main(void)
{
    unsigned int units = 250;
    long long bill_paise = calculate_bill(units);

    printf("Maximum meter reading: %u\n", MAX_READING);
    printf("Rate 1: %d paise/unit\n", RATE1_PAISE);
    printf("Rate 2: %d paise/unit\n", RATE2_PAISE);
    printf("Rate 3: %d paise/unit\n", RATE3_PAISE);

    printf("Units consumed: %u\n", units);
    printf("Bill amount: Rs %lld.%02lld\n",
           bill_paise / 100, bill_paise % 100);

    printf("sizeof(int): %zu bytes\n", sizeof(int));
    printf("sizeof(long): %zu bytes\n", sizeof(long));
    printf("sizeof(long long): %zu bytes\n", sizeof(long long));

    printf("LLONG_MAX: %lld\n", LLONG_MAX);

    return 0;
}
```

Output

```
Output

Maximum meter reading: 65535
Rate 1: 150 paise/unit
Rate 2: 200 paise/unit
Rate 3: 300 paise/unit
Units consumed: 250
Bill amount: Rs 500.00
sizeof(int): 4 bytes
sizeof(long): 8 bytes
sizeof(long long): 8 bytes
LLONG_MAX: 9223372036854775807
```

TASK-12

```
#include <stdio.h>

#define READINGS_PER_YEAR 35040
#define PEAK_MULTIPLIER 1.20

int main(void)
{
    long long yearly_wh = 0;
    long long reading_wh;
```

```
double normal_tariff;
double peak_tariff;
double total_bill = 0.0;
double units;
int i;
int hour;

printf("Enter normal tariff per kWh: ");
scanf("%lf", &normal_tariff);

printf("Enter peak tariff per kWh: ");
scanf("%lf", &peak_tariff);

printf("Enter number of readings to process: ");
scanf("%d", &i);

if (i < 1 || i > READINGS_PER_YEAR)
{
   printf("Invalid number of readings.\n");
   return 1;
}

for (int j = 0; j < i; j++)
{
   printf("Enter watt-hour reading: ");
   scanf("%lld", &reading_wh);

   printf("Enter hour (0-23): ");
   scanf("%d", &hour);

   if (reading_wh < 0 || hour < 0 || hour > 23)
   {
      printf("Invalid reading or hour.\n");
      return 1;
   }

   yearly_wh += reading_wh;
   units = reading_wh / 1000.0;

   if (hour >= 18 && hour < 22)
   {
      total_bill += units * peak_tariff;
   }
   else
   {
      total_bill += units * normal_tariff;
   }
}
```

```
    printf("\nYearly energy: %lld Wh\n", yearly_wh);
    printf("Total bill: Rs. %.2f\n", total_bill);

    return 0;
}
```

Output

```
Enter normal tariff per kWh: 5
Enter peak tariff per kWh: 8
Enter number of readings to process: 4
Enter watt-hour reading: 1000
Enter hour (0-23): 10
Enter watt-hour reading: 2000
Enter hour (0-23): 9
Enter watt-hour reading: 1500
Enter hour (0-23): 21
Enter watt-hour reading: 2500
Enter hour (0-23): 14

Yearly energy: 7000 Wh
Total bill: Rs. 39.50
```